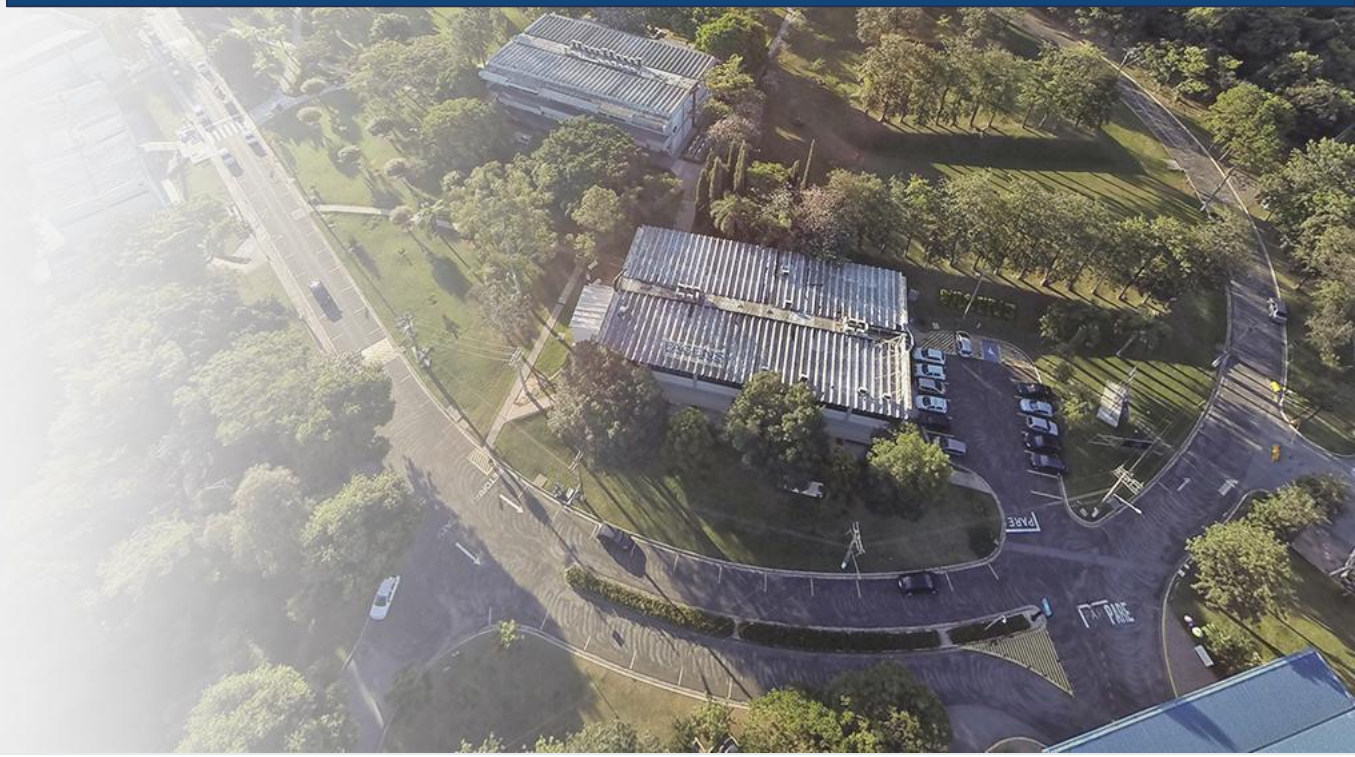
Desenvolvimento Web



Introdução ao Versionamento e Deploy

Prof. Me. Deivison S. Takatu
Email: deivison.takatu@facens.br

Sumário

- Introdução ao Versionamento e Backup
- Versionamento Semântico (SemVer)
- Git e Controle de Versão
- Branches e Merge no Git
- Deploy e Hospedagem
- Atividade.

Versionamento x Backup

Versionamento	Backup
Histórico de cada alteração	Cópia pontual do estado atual
Quem, quando e por quê mudou	Sem rastreio de autoria
Colaboração simultânea	Arquivo único, sem merge
Reversão granular	Restauração total apenas

Histórico Completo

Cada mudança é registrada com data e autor

Rastreabilidade

Saiba exatamente o que mudou e por quê

Colaboração Segura

Múltiplos devs trabalhando sem conflito

O Caos Antes do Versionamento

💾 versao_final_agora_sim2.zip

Arquivos duplicados sem nenhum contexto

⚠️ Conflito entre Devs

Dois programadores editam o mesmo arquivo

🔥 Código Perdido

Uma alteração apaga dias de trabalho

🔍 Sem Histórico

Impossível saber o que mudou e quando

Benefícios do Versionamento

Trabalho Simultâneo

Vários desenvolvedores atuam no mesmo projeto sem sobrescrever o trabalho uns dos outros.

Menos Retrabalho

Conflitos são identificados cedo, evitando perda de horas de desenvolvimento.

Auditoria e Rastreabilidade

Histórico completo de quem alterou o quê, quando e com qual justificativa.

Recuperação de Versões

Volte a qualquer ponto do histórico com segurança e sem medo de perder progresso.

Introdução ao Versionamento

- O versionamento é o processo de atribuir um identificador único a cada versão de um documento ou software.
  - Pode ser numérico (ex: v1.0, v1.1, v2.0) ou baseado em datas (ex: 2023-10-01).
  - Registra o que foi alterado, por quem e quando.
  - Garante rastreabilidade e auditoria completa das mudanças.
- Facilita a organização, colaboração e recuperação de versões anteriores.

Referências: Tate, B.; Louth, F. (2022)

MAJOR . MINOR . PATCH

1	2	3
PATCH	MINOR	MAJOR
1.0.1 Correção de bug sem quebrar nada	**1.1.0** Nova funcionalidade retrocompatível	**2.0.0** Mudança que quebra compatibilidade anterior

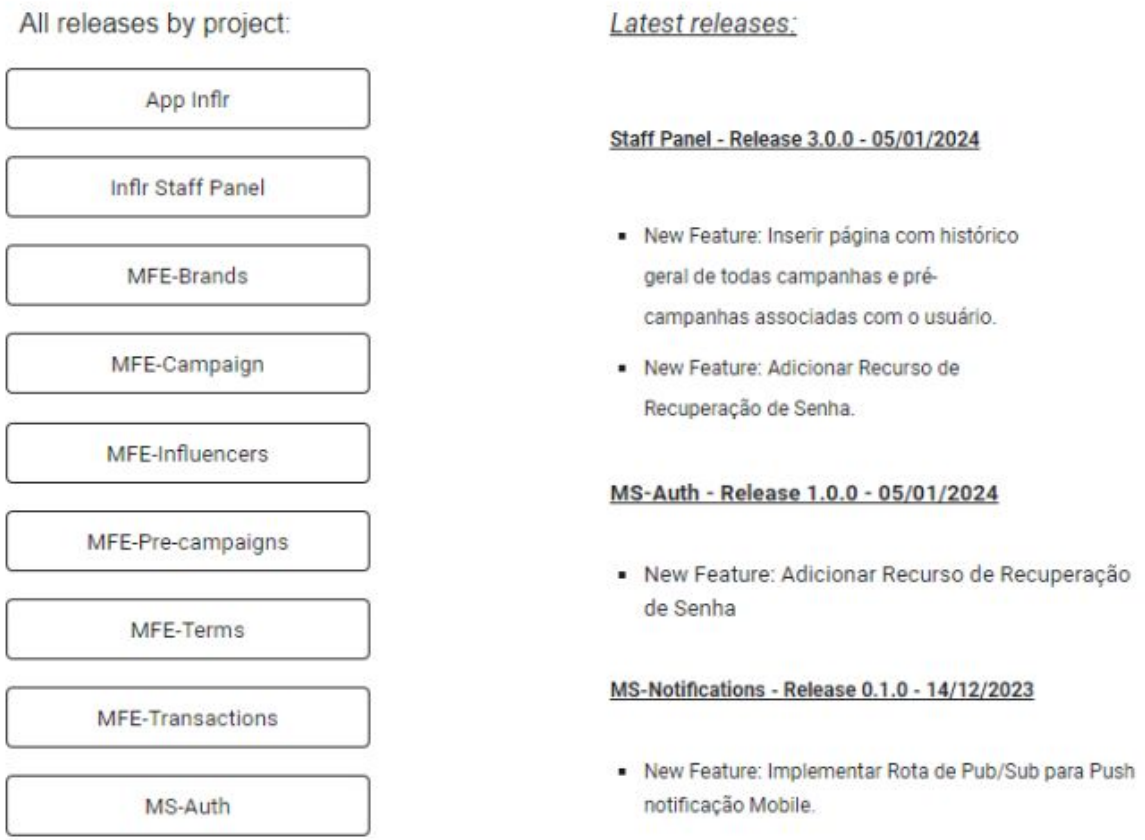
A versão **1.0.0** marca o lançamento público estável do software. Versões **0.x.x** indicam desenvolvimento inicial.

Versionamento Semântico

- Versionamento Semântico (“Semantic Versioning” ou SemVer) é um sistema de controle de versão utilizado para indicar claramente as mudanças em um software.
- As versões seguem o padrão MAJOR.MINOR.PATCH (Ex: 2.1.3), onde:
  - MAJOR (Ápice) - Mudanças incompatíveis com versões anteriores.
  - MINOR (Incremento) - Novas funcionalidades, mas compatível com versões anteriores.
  - PATCH (Correção) - Correção de bugs sem alteração na API.

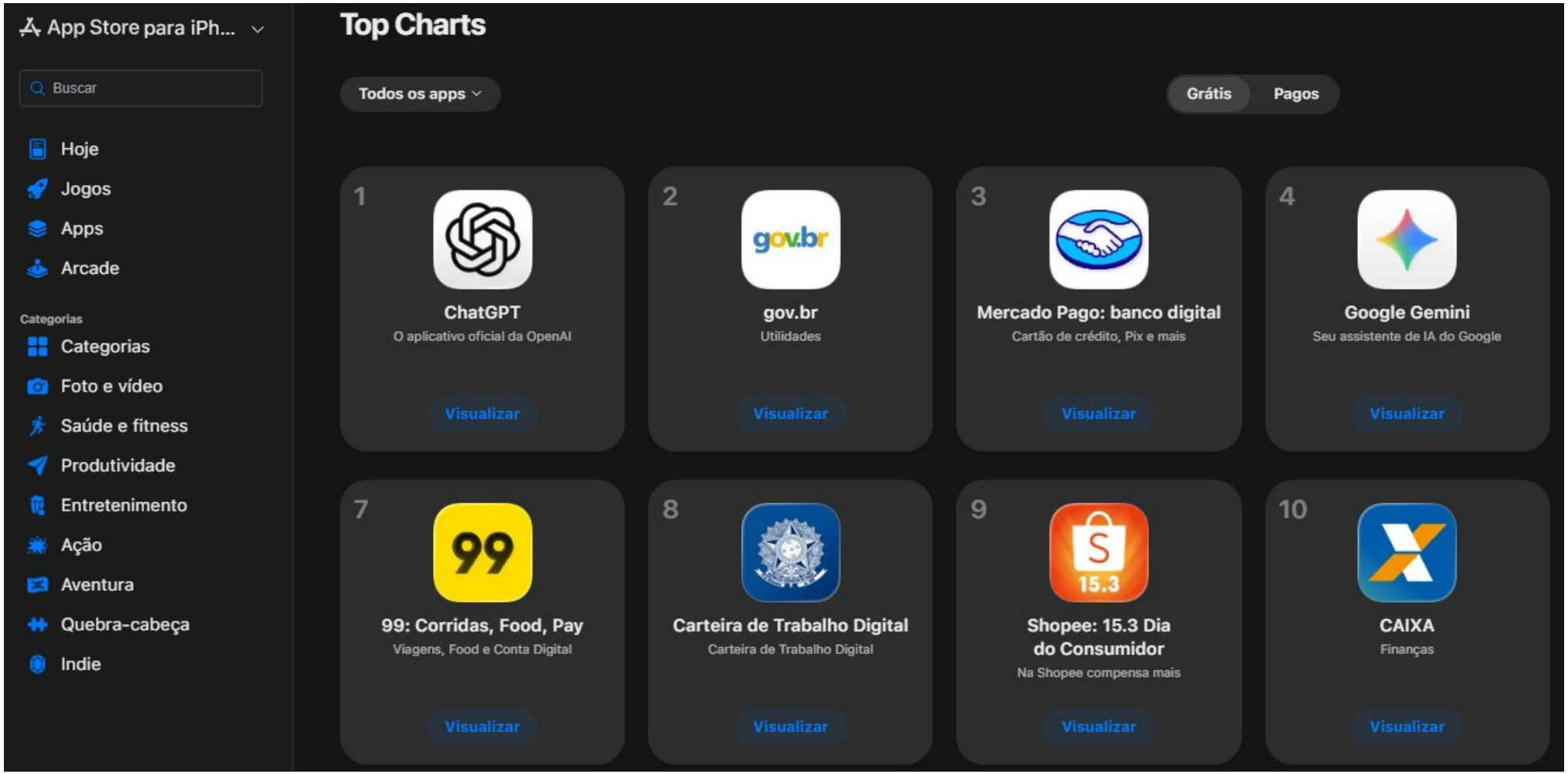
Versionamento Semântico

- Refatoração ou Nova Função: 1.0.0
- Melhorias: 0.1.0
- Bug Fix: 0.0.1



Fonte: Elaboração própria.

Versionamento Semântico



Fonte: apps.apple.com

Versionamento Semântico

ANDROID / APPS / FERRAMENTAS / GERAL / GOV.BR / **VERSÕES ANTIGAS**

Gov.br
3.8.0
Governo do Brasil

4.1
60 reviews

180.4 k
downloads

segurança

O aplicativo oficial do governo brasileiro

Versões antigas de Gov.br

Se estiver tendo problemas com a última versão de Gov.br devido a incompatibilidades com seu dispositivo, você pode baixar arquivos mais antigos do histórico de versões anteriores de Gov.br para Android na Uptodown. Enquanto você espera que o desenvolvedor de Gov.br corrija os erros da versão mais recente disponível para Android, aqui você pode baixar qualquer versão anterior de Gov.br grátis e livre de vírus. Este histórico de versões de Gov.br contém arquivos compatíveis com as diferentes arquiteturas do Android, de modo que você sempre poderá encontrar uma solução para os seus problemas.

XAPK	3.8.0 — 2 fev 2026	Android + 8.0
XAPK	3.7.50 — 1 fev 2026	Android + 8.0
XAPK	3.7.41 — 3 dez 2025	Android + 8.0
XAPK	3.7.40 — 25 jul 2025	Android + 8.0

Fonte: gov-br.br.uptodown.com

Versionamento Semântico

- Exemplo Prático:
- 1.0.0 → Primeira versão estável
- 1.1.0 → Adição de funcionalidade compatível
- 1.1.1 → Correção de bug
- 2.0.0 → Mudança incompatível com versões anteriores
- Vantagens do Versionamento Semântico:
  - Facilita a gestão de dependências.
  - Clareza para desenvolvedores e usuários.
  - Ajuda na manutenção e previsibilidade do software.

Exemplos de Alterações no Código

- Bug Fix – Correção de erros no código.
- New Feature – Adição de uma nova funcionalidade ao software.
- Feature Enhancement – Melhorias em funcionalidades existentes.
- Refactoring – Reorganização do código para torná-lo mais limpo e eficiente.
- Performance – Otimização para melhorar velocidade e eficiência.
- Security Patch – Correção de vulnerabilidades de segurança.
- Dependency Update – Atualização de bibliotecas e frameworks utilizados no projeto.
- Adding Tests – Inclusão de testes automatizados para garantir qualidade do código.

Importância do Versionamento

- Controle de Mudanças
  - Permite que múltiplos colaboradores trabalhem no projeto.
  - Facilita a integração de alterações e evita perda de trabalho.
- Histórico e Auditabilidade
  - Registra todas as modificações feitas em um arquivo.
  - Essencial para auditorias e conformidade regulatória.
- Recuperação de Dados
  - Permite reverter para versões anteriores em caso de erro.
  - Minimiza perda de trabalho e necessidade de retrabalho.
- Colaboração Eficiente
  - Melhora a comunicação entre membros da equipe.
  - Uso de branches para testar mudanças sem afetar a principal.
- Gerenciamento da Qualidade
  - Permite testar versões antes da implementação final.
  - Garante que apenas versões validadas sejam disponibilizadas.
- Aprimoramento Contínuo
  - Possibilita a análise da evolução do projeto ao longo do tempo.
  - Identifica padrões de desempenho e oportunidades de melhoria.
- Integração com Outras Ferramentas
  - Conexão com sistemas de gestão de projetos e integração contínua.
  - Centraliza o trabalho e mantém controle das mudanças.

Atividade de Revisão

Link: https://wayground.com/

WAYGROUND

formerly Quizizz

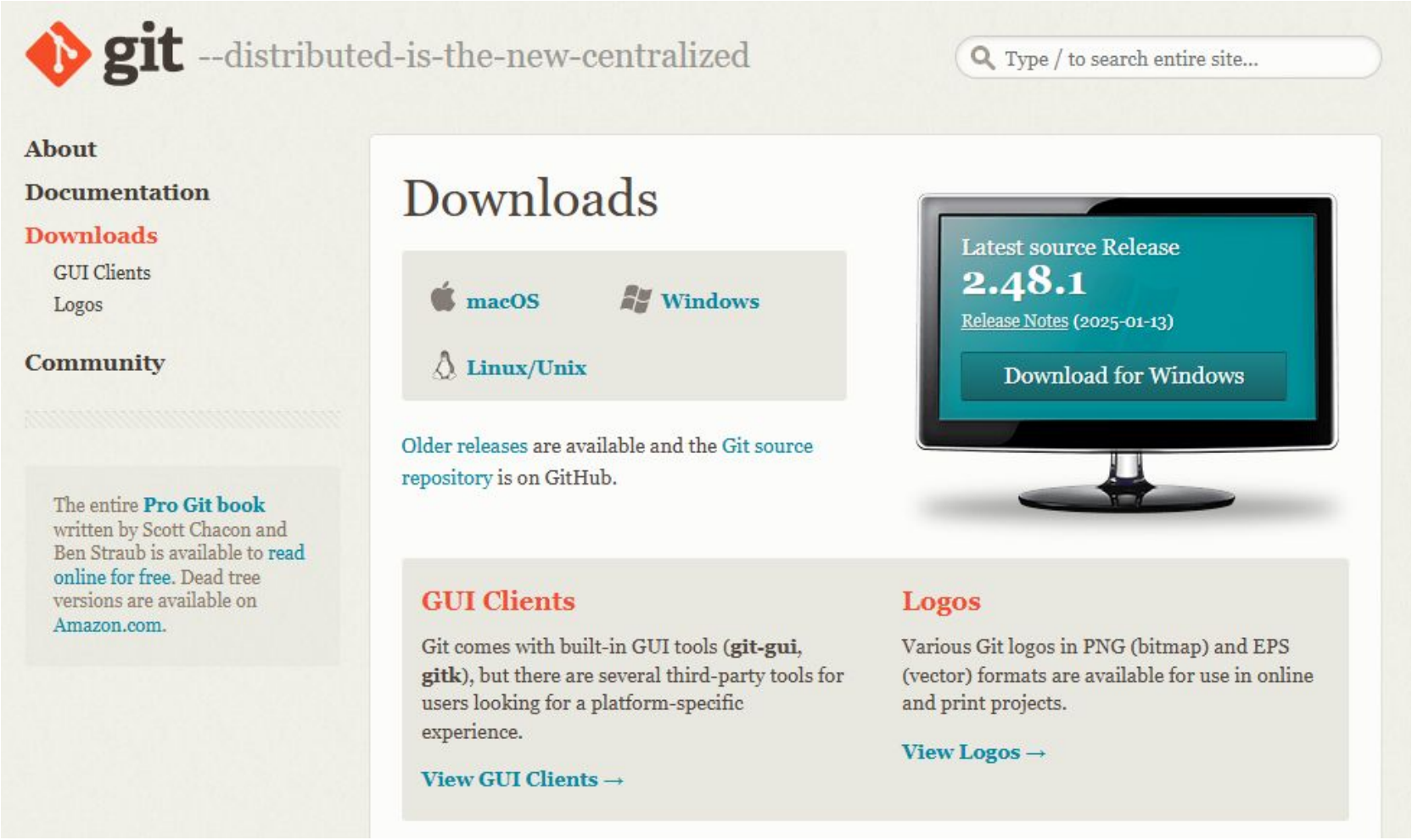
Introdução ao Git

Fonte: https://github.com/torvalds

- O que é Git?
  - Sistema de controle de versão de arquivos.
  - Instalado no computador e utilizado via linha de comando.
- O que o Git pode fazer?
  - Sincroniza com repositórios online, permitindo baixar e enviar código.
  - Registra versões do projeto, possibilitando acompanhar mudanças e restaurar versões anteriores.

Referências: Loeliger, J.; McCullough, M. (2021)

Instalando o Git



Fonte: https://git-scm.com/downloads

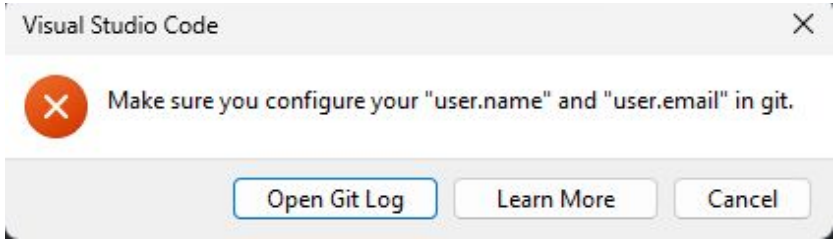
Instalando o Git

- Escolha a versão do seu sistema operacional
- Baixe e execute o instalador
- Siga as etapas "Next > Next > Install"
- Para testar a instalação:
  - Abra o Prompt de Comando
  - Digite git --version
  - Se aparecer a versão instalada, está correto

Fonte: Elaboração própria.

Instalando o Git

- Se solicitado, configure o usuário e e-mail no CMD com os comandos:
  - git config --global user.name "<Nome>"
  - git config --global user.email "<Email>"
- Abra o VS Code
- Vá até a aba de Controle de Código-Fonte (terceiro ícone à esquerda)
- Se solicitado, instale o Git
- Feche e reabra o VS Code



Fonte: Elaboração própria.

Criando um Repositório no VS Code

- Crie uma pasta no seu computador (Ex: HTML_Teste)
- Abra a pasta no VS Code
- Crie um arquivo index.html
- Vá até a aba de Controle de Código-Fonte
- Clique em Inicializar Repositório
- Adicione uma mensagem de commit e clique em Confirmar (Commit)

Publicando no GitHub

- Clique em Publicar Branch
- Faça login no GitHub
- Escolha entre repositório público ou privado
- Após a publicação, o código estará acessível no GitHub
- Acesse seu perfil no GitHub
- Vá até "Repositórios"
- Seu projeto deve estar listado
- Clique no repositório e verifique os arquivos

Tags no Git

- O que são tags?
  - Marcadores usados para identificar pontos específicos no histórico do projeto, como versões estáveis (ex: v1.0, v2.0).
- Tipos de tags:
  - Leve (lightweight): Apenas um nome para um commit específico.
  - Anotada (annotated): Armazena informações adicionais, como data, autor e mensagem.
- Uso comum: Marcar releases ou versões importantes do projeto.

Versionamento com Tags no GitHub

- O comando git tag lista as tags criadas
- Para criar uma tag, basta utilizar o comando git tag <nome da tag>
- Exemplo: git tag 1.0.0 -> poderia ser utilizada no primeiro commit
- git push origin <nome da tag>, adiciona a tag no último commit

Com isso, a cada alteração na aplicação, pode ser detalhado as alterações com maior facilidade, proporcionando uma descrição da alteração e uma sequência de versões desenvolvidas.

Situação Problema

Em uma equipe de desenvolvimento, o aplicativo em produção era mantido na branch principal (main), garantindo sua estabilidade. Surgiu a necessidade de implementar um novo recurso, ao mesmo tempo em que um bug crítico precisava ser corrigido. Para evitar riscos, a equipe criou duas branches: uma branch de feature para desenvolver o novo recurso sem interferir no sistema atual, e uma branch de hotfix para corrigir o bug urgentemente. Após testes, ambas as alterações foram integradas à branch principal, mantendo a integridade e a qualidade do aplicativo. Quais benefícios uma equipe de desenvolvimento pode ter utilizando esta abordagem durante o seu trabalho? Explique a sua resposta.

Branches no Git

- Ramificações do código que permitem trabalhar em funcionalidades ou correções sem afetar a versão principal.
  - Branch principal (main/master): Contém a versão estável do projeto.
  - Branch de desenvolvimento (develop): Usada para integrar novas funcionalidades antes de serem mescladas à main.
  - Branch de feature: Criada para desenvolver uma nova funcionalidade específica.

Merge e Conflitos no Git

- Merge: Processo de unir duas branches, integrando as mudanças de uma branch em outra.
- Conflitos: Ocorrem quando duas branches alteram a mesma parte do código; Requer intervenção manual para resolver.
- Boas práticas:
  - Sempre atualize sua branch antes de fazer merge.
  - Resolva conflitos com cuidado para evitar problemas no código.

Boas Práticas Git

- Commits pequenos e frequentes:
  - Facilita a identificação de problemas e a reversão de mudanças.
- Mensagens de commit claras:
  - Descreva o que foi alterado e por quê.
  - Uso de branches:
  - Mantenha a branch principal estável.
  - Use branches para novas funcionalidades e correções.
- Testes automatizados:
  - Garanta que o código funcione corretamente antes de fazer merge.

O que é Deploy?

- Definição: Processo de colocar uma aplicação ou software em produção, tornando-o acessível aos usuários finais.
- Objetivo: Garantir que o software funcione corretamente no ambiente de produção.
- Etapas comuns:
  - Compilação do código.
  - Configuração do ambiente.
  - Testes finais.
  - Publicação.

O que é Deploy?

Deploy é o processo de **publicar seu código em produção** — torná-lo acessível aos usuários finais. É o elo entre o desenvolvimento e o mundo real.

🖥️ Desenvolvimento

Ambiente local do dev. Erros são esperados e seguros aqui.

🧪 Staging

Cópia fiel da produção para testes finais antes do lançamento.

🚀 Produção

O ambiente real. O que os usuários acessam todos os dias.

Vercel para Deploy e Hospedagem

- Hospedagem Simplificada: Vercel é uma plataforma focada em facilitar o deploy de sites estáticos e aplicações modernas.
- Integração com Git: Conecta-se facilmente com GitHub, GitLab e Bitbucket, realizando deploys automáticos a cada push.
- Suporte a Frameworks Modernos: Ideal para aplicações desenvolvidas com Next.js, Nuxt.js, React, Vue e outros frameworks populares.
- Deploys Instantâneos: Permite rollback e deploys rápidos, garantindo atualizações ágeis e seguras.

Vercel para Deploy e Hospedagem

- Serverless Functions: Possibilita a execução de funções backend sem gerenciar servidores, escalando conforme a demanda.
- CDN Global: Distribui seu conteúdo em uma rede de alta performance, garantindo baixa latência e melhor experiência para o usuário.
- Foco em Performance e Escalabilidade: Projetada para oferecer alta performance, escalabilidade automática e segurança para aplicações web modernas.

Atividade

1. Criem um repositório no GitHub e realizem a configuração inicial de um projeto simples, como uma página HTML básica com um layout inicial. Em seguida, utilizem o Git para versionar o código e tags para marcar versões estáveis do projeto. Personalizem o projeto e realizem o deploy no Vercel, conectando o repositório do GitHub e garantindo que o site esteja acessível online.

Atividade

2. Documentem todo o processo, incluindo a justificativa das escolhas de design, as personalizações realizadas e a explicação dos passos para o versionamento e deploy. Compilem as informações em um documento organizado, com prints das telas e links do repositório no GitHub e do site no Vercel, e submetam o arquivo na plataforma CANVA.

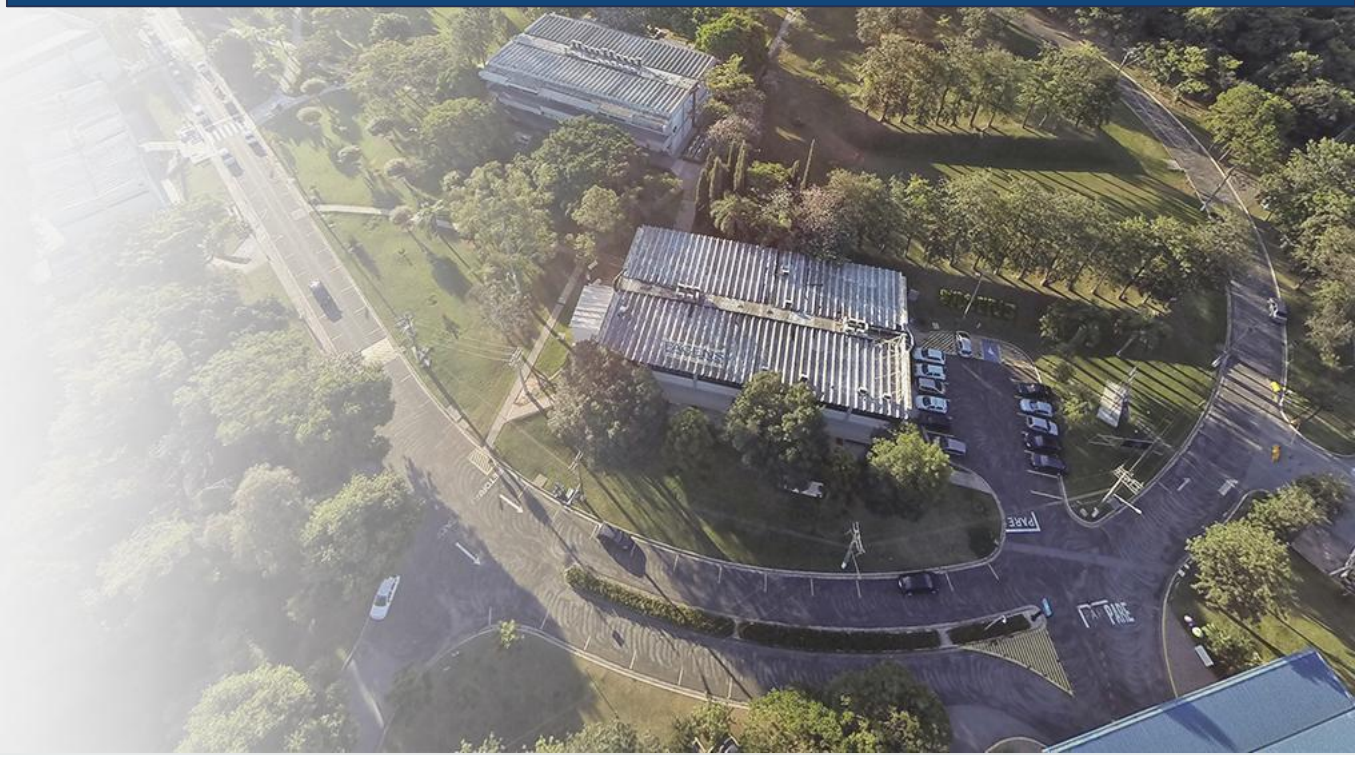
Referências

- Node.js: programe de forma rápida e prática – OLIVEIRA, Cláudio Luís Vieira. São Paulo: Expressa, 2021. Recurso online. ISBN 9786558110217.
- Interface de programação de aplicações (API) e web services – FERREIRA, Arthur Gonçalves. São Paulo: Platos Soluções Educacionais, 2021. Recurso online. ISBN 9786553560338.
- Aplicativos com Bootstrap e Angular – ZABOT, Diego. Como desenvolver apps responsivos. São Paulo: Érica, 2020. Recurso online. (Temas essenciais em mobile e websites). ISBN 9788536533049.
- Desenvolvimento de Software II – MILETTO, Evandro Manara; DE CASTRO BERTAGNOLLI, Silvia. Introdução ao Desenvolvimento Web com HTML, CSS, JavaScript e PHP. Bookman, 2014.
- Extreme Programming – WILDT, Daniel et al. Práticas para o dia a dia no desenvolvimento ágil de software. São Paulo: Casa do Código, 2015.
- JavaScript for Modern Web Development – RANJAN, Alok; SINHA, Abhilasha; BATTEWAD, Ranjit. BPB Publications, 2020.

Referências

- DINIZ, Luciana Mara Freitas et al. Aprendizado Baseado em Projetos em IHC (presencial e remoto): prototipação segundo as heurísticas de Nielsen. In: Simpósio Brasileiro de Fatores Humanos em Sistemas Computacionais (IHC). SBC, 2020. p. 13-18.
- GONÇALVES, Rodrigo Franco et al. Uma proposta de processo de produção de aplicações Web. Production, v. 15, p. 376-389, 2005.
- LUCASSEN, Garm et al. The use and effectiveness of user stories in practice. In: Requirements Engineering: Foundation for Software Quality: 22nd International Working Conference, REFSQ 2016, Gothenburg, Sweden, March 14-17, 2016, Proceedings 22. Springer International Publishing, 2016. p. 205-222.
- SALINAS, C.; SEDEÑO, J.; CUARESMA, M.; RISOTO, M. Agile, Web Engineering and Capability Maturity Model Integration: A systematic literature review. Information and Software Technology, v. 71, p. 92-107, 2016. DOI: 10.1016/J.INFSOF.2015.11.002.

Desenvolvimento Web



Introdução ao Versionamento e Deploy

Prof. Me. Deivison S. Takatu
Email: deivison.takatu@facens.br